CHEAPESTGO · V2 MIGRATION · FRONTEND REFERENCE

Frontend Gap — Implementation Plan

All UI components and pages present in V1 (cheapest-go-app) that are absent from V2 (cheapestgo-app-v2), with step-by-step implementation guidance, file targets, and API dependencies.

40+ total gaps | 2 Tier 1 — Critical | 3 Tier 2 — Significant | 5 Tier 3 — Enhancement

Next.js 15 App Router | cheapestgo-app-v2

TIER 1 — CRITICAL Checkout & Cancellation

Missing components break core conversion & refund flows

FE-01 src/app/checkout/ — Checkout components

8 missing · V1 has 12 · V2 has 3

Voucher input, promo list, Stripe embedded, booking success, policies, special requests all absent

MISSING COMPONENTS

- VoucherInput.tsx
- AvailablePromos.tsx
- StripeEmbeddedCheckout.tsx
- BookingSuccess.tsx
- CancellationPolicySection.tsx
- SpecialRequestsSection.tsx
- BookingForSection.tsx
- SubmitBookingButton.tsx

FILES

- src/app/checkout/components/VoucherInput.tsx
- src/app/checkout/components/AvailablePromos.tsx
- src/app/checkout/components/StripeEmbeddedCheckout.tsx
- src/app/checkout/components/BookingSuccess.tsx
- src/app/checkout/components/CancellationPolicySection.tsx
- src/app/checkout/components/SpecialRequestsSection.tsx
- src/app/checkout/components/BookingForSection.tsx
- src/app/checkout/components/SubmitBookingButton.tsx

IMPLEMENTATION STEPS

1. **VoucherInput** — text field + "Apply" button. On submit: `POST /api/vouchers/validate` with `{ code, totalAmount }`. On success show discount badge and subtract from displayed total via parent state setter. Show inline error on invalid code.
2. **AvailablePromos** — fetch `GET /api/vouchers/list` on mount. Render each active voucher as a clickable chip showing code + discount. Clicking fires the same validate flow as VoucherInput and auto-fills the code field.
3. **StripeEmbeddedCheckout** — import `EmbeddedCheckout` from `@stripe/react-stripe-js`. Receive `clientSecret` prop (obtained from `POST /api/hotels/create-payment` before this component mounts). Wire `onComplete` to call `POST /api/hotels/confirm` and redirect to `/checkout/success?bookingId=`.
4. **BookingSuccess** — rendered at `/checkout/success`. Show: booking reference, property name, check-in/out dates, guest count, total paid. Download invoice button → `GET /api/invoice/:id/pdf?type=hotel`. Share button (navigator.share). "View my trips" CTA.
5. **CancellationPolicySection** — receive `refundableTag` and `free_cancel_deadline` from prebook response props. Render a green "Free cancellation until {date}" banner when refundable, or an amber "Non-refundable" badge when not. Include the full policy text in a collapsed accordion.
6. **SpecialRequestsSection** — `<textarea>` for dietary / accessibility notes. Bound to form state; value passed to `POST /api/hotels/confirm` as `specialRequests`. Character counter (max 500).
7. **BookingForSection** — radio: "Booking for myself" vs "Booking for someone else". When "someone else" selected, reveal guest name + email fields. Populate `guestName` / `guestEmail` in booking payload.
8. **SubmitBookingButton** — disabled while any required field is empty or form is submitting. Shows spinner during submission. On Stripe redirect-based flows, replaces itself with `StripeEmbeddedCheckout` once clientSecret is ready.

API DEPENDENCIES

`POST /api/vouchers/validate` `GET /api/vouchers/list` `POST /api/hotels/create-payment` `POST /api/hotels/confirm` `GET /api/invoice/:id/pdf`

●●●●○ High complexity — @stripe/react-stripe-js · zustand · react-hook-form

FE-02 **src/app/trips/ — Cancellation flow + Flight card** — 6 missing · V1 has 8 · V2 has 3

Self-service cancel, fee display, modification modal, trip map, flight booking card all absent

MISSING COMPONENTS

`CancellationModal.tsx` `CancellationFeeCard.tsx` `CancellationPolicies.tsx` `ModificationModal.tsx` `TripMapView.tsx` `FlightBookingCard.tsx`

FILES

`src/app/trips/components/CancellationModal.tsx`

`src/app/trips/components/CancellationFeeCard.tsx`

`src/app/trips/components/CancellationPolicies.tsx`

`src/app/trips/components/ModificationModal.tsx` `src/app/trips/components/TripMapView.tsx`
`src/app/trips/components/FlightBookingCard.tsx`

IMPLEMENTATION STEPS

1. **CancellationModal** — two-step dialog. Step 1: for flights, call `POST /api/flights/cancel-quote` to get the refund/penalty breakdown; for hotels, pull the stored cancellation policy. Display via `CancellationFeeCard`. Step 2: user presses "Confirm cancellation" → call `POST /api/flights/cancel-booking` or `POST /api/hotels/cancel`. On success, invalidate trips query and show a success toast.
2. **CancellationFeeCard** — display original amount, penalty, and net refund side-by-side. Color: refund amount in green, penalty in red. Show "Full refund" badge when penalty is 0. Show "Non-refundable" when refund is 0. Used as a child of CancellationModal step 1.
3. **CancellationPolicies** — static display of the policy text from the booking record. Formats each policy deadline as a human-readable line: "Cancel before {date}: {amount} penalty". Used on the trip detail page below the booking summary.
4. **ModificationModal** — for date changes only (V1 scope). Renders a date range picker pre-filled with current check-in/out. On submit: call `POST /api/hotels/modify` if the endpoint exists; otherwise show "Contact support" fallback. Disable submit if dates unchanged.
5. **TripMapView** — `dynamic(() => import('@/components/map/SearchMapContainer'), { ssr: false })`. Pass destination coordinates from the booking record. Shows a single destination pin + nearby POI gems. Renders at the bottom of the trip detail page below the booking summary card.
6. **FlightBookingCard** — card variant for `flight_bookings` records. Show: airline logo, origin → destination, departure/arrival times, cabin class badge, PNR, status chip (confirmed / ticketed / cancelled). Differentiated from the hotel `BookingCard` by icon and layout. Include download e-ticket button when status is `ticketed`.

API DEPENDENCIES

`POST /api/flights/cancel-quote` `POST /api/flights/cancel-booking` `POST /api/hotels/cancel` `GET /api/trips/:id`

●●●○○ Medium complexity — zustand · mapbox-gl

TIER 2 — SIGNIFICANT

Property, Search & Flights

User-visible feature gaps — incomplete pages without these

FE-03 **`src/app/property/[id]/ — Property page`** — 14 missing · V1 has ~22 · V2 has 5

Booking widget, room cards, mobile CTA, policies, FAQ, weather, reviews modal, location section all absent

MISSING COMPONENTS (IMPLEMENT IN THIS ORDER)

`BookingWidget.tsx` `PropertyDatePicker.tsx` `MobileBookingCTA.tsx` `MobilePropertyHeader.tsx`

`RoomCard.tsx` `RoomsAvailabilitySection.tsx` `PropertyNav.tsx` `PoliciesSection.tsx` `FAQSection.tsx` `WeatherWidget.tsx` `ReviewsModal.tsx` `ReviewsSummary.tsx` `LocationSection.tsx` `PoiDetailsModal.tsx`

IMPLEMENTATION STEPS

1. **BookingWidget** (desktop sidebar) — `position: sticky; top: 80px`. Show price-per-night, date range trigger (opens PropertyDatePicker), room/guest selectors, subtotal, and a "Reserve" CTA that navigates to `/checkout?propertyId=&rateKey=&checkIn=&checkOut=&rooms=`. Disable CTA until dates are selected.
2. **PropertyDatePicker** — inline calendar rendered inside BookingWidget and in the mobile modal. Reads/writes check-in + check-out from `useSearchStore`. Blocks past dates. Shows a 2-month panel on desktop, 1-month on mobile. On date selection, triggers a re-fetch of room availability.
3. **MobileBookingCTA** — fixed bottom bar (`position:fixed;bottom:0;left:0;right:0;z-index:50`). Show price/night on the left, "Reserve" button on the right. Hidden on desktop via `lg:hidden`. Tapping "Reserve" opens a bottom sheet containing PropertyDatePicker + room selector + BookingWidget CTA.
4. **RoomsAvailabilitySection** — on mount (or when dates change) call `POST /api/hotels/prebook` with `{ propertyId, checkIn, checkOut, rooms }`. Render a `RoomCard` for each rate in the response. Show skeleton cards while loading. Show "No rooms available" state if empty.
5. **RoomCard** — display: room name, bed type, max occupancy, board basis badge (e.g. "Breakfast included"), refundable badge, price per night, total price, "Select room" button. Clicking sets the selected `rateKey` in checkout store and scrolls to BookingWidget.
6. **PropertyNav** — sticky anchor bar (Overview · Rooms · Reviews · Location · Policies). Appears below the hero gallery. Uses `IntersectionObserver` on each section's id to highlight the active tab as user scrolls. Clicking a tab smoothly scrolls to that section.
7. **WeatherWidget** — call `GET /api/weather?lat={lat}&lng={lng}` using the property's `latitude` / `longitude` fields. Show current temperature + weather icon + 5-day forecast row. Cache result in component state for the session. Display "Weather unavailable" if API fails.
8. **PoliciesSection** — render property `policies` array from the property detail response. Accordion per policy type (check-in, check-out, pets, children, smoking). Use `details/summary` HTML elements for zero-JS accordion.
9. **FAQSection** — same accordion pattern as PoliciesSection. Hardcoded FAQ items (e.g. "How do I cancel?", "Is early check-in available?") supplemented by any property-specific FAQ from the API response.
10. **ReviewsModal + ReviewsSummary —** Summary: show overall score, rating distribution bar chart (1–5 stars), top categories (cleanliness, location, value). Modal: paginated reviews list (20/page). Open from "See all {N} reviews" button. Fetch from `GET /api/hotels/:id/reviews`.
11. **LocationSection** — embed a small static Mapbox map centered on property coordinates. List nearby POIs (airport distance, metro, attractions) from property data. "Open in Google Maps" link.

API DEPENDENCIES

`POST /api/hotels/prebook` `GET /api/weather` `GET /api/hotels/:id/reviews` `GET /api/photos/hotel`

●●●●○ High complexity mapbox-gl · zustand · date-fns

FE-04 **src/app/search/ — Map view + Filters** 8 missing · V1 has 17 · V2 has 3

Map/list split view, search filters panel, mobile search modal, loading skeleton all absent

MISSING COMPONENTS

`SearchMapView.tsx` `MapResultsClient.tsx` `SearchFilters.tsx` `ActiveFiltersSummary.tsx` `CountryCityPicker.tsx` `MobileSearchModal.tsx` `SearchLoadingSkeleton.tsx` `ResponsiveSearchHeader.tsx`

IMPLEMENTATION STEPS

1. **SearchMapView** — split-panel layout: left = scrollable hotel card list (existing `HotelResultsClient`), right = sticky `SearchMapContainer`. On mobile, toggle between panels with a "Map" / "List" button. Wrap map in `dynamic(() => ..., { ssr: false })`. Port directly from V1's `SearchMapView.tsx` — it is fully implemented there.
2. **MapResultsClient** — client component that owns the SSE subscription to `POST /api/hotels/search/stream`, accumulates hotel results, and passes them to both the list and map via context. Replaces the current page-level fetch pattern.
3. **SearchFilters** — sidebar/drawer with: price range slider (`min` / `max` from search results), star rating checkboxes (1–5), board type chips (Room Only / Breakfast / All-Inclusive), refundable toggle, property type chips. All state lives in `useSearchStore.filters`. Filters are applied client-side on the accumulated results array.
4. **ActiveFiltersSummary** — horizontal row above results showing one removable chip per active filter (e.g. "★★★★★ ×", "Breakfast ×"). Clicking × removes that filter from store. "Clear all" link at the end. Hidden when no filters active.
5. **SearchLoadingSkeleton** — grid of 6 animated pulse cards matching the hotel card dimensions. Show while SSE stream is open and `results.length === 0`. Replaced by actual cards as results stream in.
6. **Map/List toggle** — add `?view=map|list` URL param. Search page reads param to conditionally render `SearchMapView` (map+list) vs plain list. Toggle button updates the URL param via `router.push`.

API DEPENDENCIES

`POST /api/hotels/search/stream (SSE)` `GET /api/photos/destination`

●●●●○ High complexity mapbox-gl · zustand · EventSource

FE-05 **src/app/flights/ — Advanced booking components** 5 missing · V1 has 13 · V2 has 8

Seat map, bag selector, price calendar, price alert button, fare conditions absent

MISSING COMPONENTS

`BagSelectionPanel.tsx` `SeatMapPanel.tsx` `PriceCalendar.tsx` `PriceAlertButton.tsx` `DuffelFareConditions.tsx`

IMPLEMENTATION STEPS

1. **BagSelectionPanel** — shown on the flight booking page. On mount, call `POST /api/flights/bags` with `{ offerId }`. Group returned services by type (carry-on, checked 23kg, checked 32kg). Render each as a quantity stepper (`-/+`). Sum selected services' prices and add to displayed total. Persist selection to booking payload as `services: [{ id, quantity }]`.
2. **SeatMapPanel** — call `POST /api/flights/seat-map` with `{ offerId }`. Render each cabin section as a grid: available seats are clickable buttons, occupied seats are greyed out divs, exit rows get a "EXIT" label. Color-code by type (standard / extra-legroom / front). Selected seat shown in blue. Seat selection stored in booking payload as `{ segmentId, passengerId, designator }`.
3. **PriceCalendar** — shown above search results on the flights page. Fetch `GET /api/flights/price-calendar?origin&destination&month&cabin`. Render a calendar grid with price labels below each date. Color-code cheapest 20% of days in green, most expensive 20% in red. Clicking a date updates the departure date search param and re-runs the search.
4. **PriceAlertButton** — toggle button in the flight search results header. When enabled: `POST /api/bookings/price-alerts` with `{ origin, destination, cabinClass, targetPrice }`. Show a confirmation toast. On page load for an authenticated user, check `GET /api/bookings/price-alerts?route=` to pre-set the toggle state. Disabled for unauthenticated users with tooltip "Sign in to set price alerts".
5. **DuffelFareConditions —** collapsible panel below the fare selector on the booking page. Fetch `POST /api/flights/fare-rules` with `{ offerId }`. Display cancellation penalty tiers and change fee conditions in a simple two-column table. Collapsed by default; expand on "View fare rules" click.

API DEPENDENCIES

`POST /api/flights/bags` `POST /api/flights/seat-map` `GET /api/flights/price-calendar` `POST /api/bookings/price-alerts` `POST /api/flights/fare-rules`

●●●○○ Medium complexity — zustand · react-query

TIER 3 — ENHANCEMENT

Auth Modal, Admin, UI Primitives & Landing

Lower user impact — ship after Tier 1 & 2

FE-06 `src/components/auth/` — In-page auth modal flow

8 missing · V1 has 13 · V2 has 4

V2 currently hard-navigates to /login. V1 showed a multi-step modal without leaving the page.

MISSING COMPONENTS

`AuthModal.tsx` `AuthModalWrapper.tsx` `EmailStep.tsx` `PasswordStep.tsx` `RegisterStep.tsx` `ForgotPasswordStep.tsx` `VerifyEmailStep.tsx` `PasswordRequirements.tsx`

IMPLEMENTATION STEPS

1. Add `useAuthModal` Zustand slice: `{ isOpen, step, email, open(step?), close() }`. Persist `isOpen` as local state only (not localStorage).

2. **AuthModal** — full-screen overlay on mobile, centered dialog on desktop. Renders the active step component based on `useAuthModal().step`: `'email' | 'password' | 'register' | 'forgot' | 'verify'`.
3. **EmailStep** — single email field. On submit: check if email exists via `POST /api/auth/check-email`. If yes → navigate to `PasswordStep`. If no → navigate to `RegisterStep`. Pre-fill email in both.
4. **PasswordStep** — password field + "Sign in" button. On success call Supabase `signInWithPassword`. On error show inline message. "Forgot password?" link navigates to ForgotPasswordStep.
5. **RegisterStep** — name + password fields + `PasswordRequirements` indicator. On submit call Supabase `signUp`. On success navigate to VerifyEmailStep.
6. Replace all `router.push('/login')` calls site-wide with `useAuthModal().open()`. Search: ~12 occurrences in header, booking CTAs, price alert button, wishlist button.

●●●○○ Medium complexity — supabase-js · zustand · @supabase/ssr

FE-07 **src/app/admin/ — Charts & Command Palette** — 8 missing · V1 has 14 · V2 has 3

Revenue chart, Duffel insights, funnel, top routes, command palette, top nav all absent

MISSING COMPONENTS

`CommandPalette.tsx` `RevenueChart.tsx` `DuffelInsightsCharts.tsx` `ConversionFunnel.tsx` `ProviderIntegrations.tsx` `TopRoutes.tsx` `TopNav.tsx` `MobileAdminNav.tsx`

IMPLEMENTATION STEPS

1. Install `recharts` (or reuse if already present). All charts use `ResponsiveContainer` for fluid width.
2. **RevenueChart** — `LineChart` from `GET /api/admin/stats?range=30d`. X-axis: date, Y-axis: revenue (PHP). Two series: hotels (blue) + flights (orange). Include a date-range toggle (7d / 30d / 90d).
3. **DuffelInsightsCharts** — `BarChart` for booking counts by route. Data from `GET /api/admin/stats?type=routes`. Separate bar groups for economy vs business cabin.
4. **ConversionFunnel** — horizontal funnel: Search → Prebook → Checkout → Confirmed. Each stage shows count + conversion %. Data from `GET /api/admin/stats?type=funnel`.
5. **CommandPalette** — keyboard shortcut `Ctrl+K` (or `⌘K`). Overlay with search input filtering admin nav links. Arrow key navigation + Enter to navigate. Dismiss on Escape or backdrop click. Fully client-side — no API call needed.
6. **TopNav** — top bar with: logo, global search trigger (→ CommandPalette), notification bell (badge from unread count), user avatar menu. **MobileAdminNav**: bottom tab bar for mobile admin with icons for Dashboard / Bookings / Users / Settings.

API DEPENDENCIES

`GET /api/admin/stats` `GET /api/admin/notifications`

●●○○○ Low–Medium — recharts · zustand

FE-08 **src/components/ui/ + common/ — UI Primitives & Utilities** 13 missing across ui/ and common/

Standard shadcn components + small utilities that are used across many pages

MISSING FROM SRC/COMPONENTS/UI/

`alert-dialog.tsx` `avatar.tsx` `dropdown-menu.tsx` `sheet.tsx` `table.tsx` `logo.tsx` `optimized-image.tsx` `animations.tsx`

MISSING FROM SRC/COMPONENTS/COMMON/

`BackButton.tsx` `ClientOnly.tsx` `CurrencySelector.tsx` `FormDatePicker.tsx` `SaveButton.tsx`

IMPLEMENTATION STEPS

1. Scaffold shadcn components in one command: `pnpm dlx shadcn@latest add alert-dialog avatar dropdown-menu sheet table`
2. **logo.tsx** — SVG or img tag wrapping the CheapestGo logo asset. Accepts `size` prop. Port from V1's `src/components/ui/logo.tsx`.
3. **optimized-image.tsx** — thin wrapper around `next/image` with default placeholder blur, fallback src on error, and a `fill` mode helper prop. Port from V1.
4. **BackButton** — `"use client"`. Calls `router.back()`. If `href` prop provided, falls back to `router.push(href)` when there's no history. Used on property, checkout, and trip detail pages.
5. **ClientOnly** — renders `null` until after first hydration. Pattern: `const [mounted, setMounted] = useState(false); useEffect(() => setMounted(true), []); if (!mounted) return null;`. Wraps Mapbox and any window-dependent components to prevent SSR mismatch.
6. **CurrencySelector** — dropdown of supported currencies (PHP, USD, SGD, AUD, GBP, EUR). Writes to `useUserPreferences().currency` store and POSTs to `/api/users/preferences` for authenticated users. Persisted to `localStorage` for guests.

●●○○○ Low complexity shadcn/ui · next/image

FE-09 **src/app/ (landing) — Missing landing sections & AI search** 10 missing components

5 display-only landing sections + entire AI search overlay absent. Landing sections can ship now; AI search blocked on voice API.

MISSING LANDING SECTIONS (SHIP NOW — NO API BLOCKERS)

`CuratedSection.tsx` `ExploreUniqueStays.tsx` `GuestFavoritesSection.tsx` `HotelDealsSection.tsx` `LastMinuteWeekendDeals.tsx`

MISSING AI SEARCH (BLOCKED ON POST /API/VOICE)

`AISearchBar.tsx` `AIPromptInput.tsx` `AIResultsPreview.tsx` `SearchModeToggle.tsx` `AISuggestionChips.tsx`

IMPLEMENTATION STEPS

1. **Landing sections** — all are display-only. Port directly from V1 source files. No API changes needed; they consume existing `GET /api/hotels/deals` and `GET /api/hotels/popular` endpoints. Add each as a `<section>` in the landing page below the hero.
2. **HotelDealsSection** — fetch `GET /api/hotels/deals?limit=6`. Render horizontal scroll of deal cards with original vs discounted price. Port from V1.
3. **GuestFavoritesSection** — fetch `GET /api/hotels/popular?type=favorites&limit=8`. Render a 4-column grid of property cards with rating badges. Port from V1.
4. **AI search components** — defer until `POST /api/voice` is implemented in the API. When the API is ready, port `AISearchBar.tsx` → `AIPromptInput` → streaming `AIResultsPreview` from V1. Wire `SearchModeToggle` to swap between standard search and AI search mode.

API DEPENDENCIES (LANDING SECTIONS)

`GET /api/hotels/deals` `GET /api/hotels/popular`

●●○○○ Low complexity (sections) · High (AI search) — Port directly from V1 for sections

EXCLUDED

Out of Scope

Intentionally deferred or not required for V2

ITEM	REASON
`VoiceAssistant.tsx`	Depends on `POST /api/voice` — not in V2 API scope yet.
`AISearchBar + overlay`	Blocked on voice API. Defer until voice is live.
`/auth/auth-code-error page`	Redirect to `/login?error=oauth_failed` in middleware instead — no dedicated page needed.
`MapboxDirections / TripRouteLayer / ClusterLayer`	Needed only for itinerary trip-planning feature — not on the critical path for V2 launch.
`GpsMarker / useGpsTracking / useGoogleSearch`	Mobile-specific or itinerary-specific — defer.
`MagneticButton / TiltCard / animations.tsx flourishes`	Animation enhancement only — ship after functional parity is reached.
`/dev-policy-test / /meta-preview / /test-map`	Dev utility pages — add locally when needed, do not ship to production.
`PlaceImage / PlaceList`	Used only by the places/ feature which is not scoped for V2.

Generated 2026-07-09 · CheapestGo Frontend Gap Analysis · cheapestgo-app-v2 · 40+ gaps across 5 tiers